Compte-Rendu Projet MU4IN508

Observing Linear Hashing : Optimality and Applications

Encadré par : Ky Nguyen

Kairui PENG 21219046

Shuoliang YANG 21500629

Feiyang CHU 21208393

Année universitaire 2025–2026

Table des matières

1 Introduction **3**

2 Preliminaries and Notations **3**

3 Linear Hashing Model **5**

4 Experimental Setup **6**

5 Results and Profiling **10**

6 Discussion **10**

7 Conclusion **10**

Travail réalisé

Dans ce cadre, nous avons :

— implémenté la famille des fonctions de hachage linéaires définies par $h(x) = Ax \bmod 2$;
— évalué expérimentalement la charge maximale pour différents paramètres (m, n) ;
— comparé les résultats expérimentaux aux bornes théoriques établies pour la charge maximale ;
— optimisé l'implémentation en remplaçant la version Python/NumPy par une version C++, afin de réduire les coûts temporels et mémoire ;
— analysé les limites pratiques des simulations lorsque ℓ devient grand.

Les expériences confirment qualitativement le comportement théorique dans le régime accessible computationnellement. Cependant, lorsque les paramètres approchent le régime asymptotique, le coût de calcul devient prohibitif.

Lien vers Github :https ://github.com/C-killer/Observing-Linear-Hashing ?tab=readme-ov-file Utilisez le package `url` pour embedded links

Travail restant

Conformément à la description officielle du projet les deux parties suivantes restent à implémenter.

1. Prototype de schémas de signature basés sur le hachage linéaire **Objectif :** Implémenter un prototype minimal de schéma de signature ou multi-signature basé sur les fonctions de hachage linéaires

Tâches à réaliser :

— Étudier et identifier le rôle précis du hachage linéaire dans le schéma.
— Définir les interfaces : `KeyGen`, `Sign`, `Verify`.
— Implémenter un prototype minimal fonctionnant sur une seule machine (multi-processus suffisant).
— Mesurer le temps de signature, de vérification, et le coût de communication.

Délai prévu : avant le 15 avril

2. (Option trinôme) Profilage en norme ℓ_∞ **Objectif :** Profiler expérimentalement le résultat concernant les garanties en norme ℓ_∞.

Tâches à réaliser :

— Comprendre les paramètres (u, ℓ, τ, δ) et la condition sur la perte d'entropie.
— Estimer empiriquement, pour tout $y \in \mathbb{F}_2^\ell$:

$$\left|\Pr[h(U_S) = y] - 2^{-\ell}\right|.$$

— Comparer la valeur maximale obtenue avec la borne théorique $\tau 2^{-\ell}$.
— Étudier graphiquement l'impact des différents paramètres.

Délai prévu : avant le 6 mai

Ky : à ajouter un 3e point, e.g. "rédiger rapport et documenter le travail dans Git"

3. Rédiger rapport et documenter le travail dans Git

1 Introduction

Ajouter bibliography pour les citations ?

L'utilisation des fonctions de hachage est omniprésente dans la conception et l'analyse des algorithmes aléatoires. En 1997, Alon, Dietzfelbinger, Miltersen, Petrank et Tardos ont posé la question : « Is Linear Hashing Good ? ». L'article *Linear Hashing Is Optimal* de Michael Jaber, Vinayak M. Kumar et David Zuckerman apporte une réponse à cette question par une démonstration mathématique : une matrice aléatoire sur le corps $\mathbb{F}_2$ est utilisée pour répartir n boules dans n bacs, et la charge maximale $L_{\max}$ vérifie

$$\mathbb{E}[L_{\max}] = O\left(\frac{\log n}{\log \log n}\right).$$

L'un des objectifs principaux de ce projet est de vérifier expérimentalement la validité de ce résultat théorique à travers une implémentation et des expériences informatiques.

En particulier, nous étudions le comportement de la famille la plus simple possible de fonctions de hachage : les matrices aléatoires dans un corps de caractéristique 2, i.e. $\mathbb{F}_2$:

— Soit $\ell := \log n$ et nous organisons nos n urnes en un espace vectoriel de dimension ℓ sur $\mathbb{F}_2$, c'est-à-dire l'espace vectoriel $\mathbb{F}_2^\ell$.

— L'ensemble univers à hacher est l'espace vectoriel $\mathbb{F}_2^u$.

— Notre famille de fonctions de hachage contient des applications linéaires

$$h : \mathbb{F}_2^u \to \mathbb{F}_2^\ell.$$

Notre projet se divise en trois parties :

1. Une implémentation de familles de fonctions de hachage linéaires sur $\mathbb{F}_2$, accompagnée d'un profilage, afin de démontrer expérimentalement la charge maximale moyenne optimale théoriquement prouvée pour mapper m balles vers n urnes.
2. Un prototype de divers schémas de signature basés sur des fonctions de hachage linéaires, généralisées heuristiquement sur des corps de caractéristique autre que 2.
3. Comme extension des points de travail ci-dessus, nous pouvons compléter ce profilage des propriétés du hachage linéaire en examinant son comportement en termes de normes au-delà de la norme ℓ_1.

2 Preliminaries and Notations

Cadre algébrique

Nous travaillons sur le corps fini

$$\mathbb{F}_2 = \{0, 1\},$$

muni de l'addition et de la multiplication modulo 2. En particulier, l'addition dans $\mathbb{F}_2$ correspond à l'opération XOR. Toutes les opérations vectorielles et matricielles considérées dans ce rapport sont effectuées modulo 2.

Pour un entier $u \geq 1$, on note

$$\mathbb{F}_2^u$$

l'espace vectoriel des vecteurs binaires de dimension u. Un vecteur $x \in \mathbb{F}_2^u$ s'écrit

$$x = (x_1, \ldots, x_u), \qquad x_i \in \{0, 1\}.$$

Poids de Hamming

Le *poids de Hamming* d'un vecteur $x \in \mathbb{F}_2^u$ est défini par

$$\|x\|_1 = \sum_{i=1}^{u} x_i.$$

Il représente le nombre de coordonnées non nulles (i.e., égales à 1) dans x.

Distributions d'entrée

On considère des entrées aléatoires

$$X \sim \mathcal{D},$$

à valeurs dans $\mathbb{F}_2^u$, sous plusieurs distributions $\mathcal{D}$.

Afin d'obtenir une analyse plus complète et d'éviter que les observations expérimentales ne soient dues à un cas particulier, nous considérons à la fois des distributions à coordonnées indépendantes et des distributions introduisant des corrélations entre bits.

Plus précisément, nous utilisons deux modèles à bits indépendants et deux modèles à dépendances structurées, ce qui permet d'évaluer l'impact de l'indépendance — ou de son absence — sur le comportement du hachage linéaire.

Distribution uniforme. Chaque coordonnée est générée indépendamment selon

$$X_i \sim \text{Bernoulli}(1/2).$$

Ainsi, tous les vecteurs de $\mathbb{F}_2^u$ sont équiprobables.

Distribution Bernoulli biaisée (p-biased). Pour un paramètre $p \in (0, 1)$, les coordonnées sont indépendantes et

$$X_i \sim \text{Bernoulli}(p).$$

Cette distribution conserve l'indépendance des bits mais n'est pas symétrique lorsque $p \neq 1/2$.

Distribution à poids fixé. Pour un entier $k \in \{0, \ldots, u\}$, on échantillonne uniformément parmi les vecteurs vérifiant

$$\|x\|_1 = k.$$

Dans ce cas, les coordonnées sont fortement corrélées.

Distribution markovienne. Les bits sont générés séquentiellement avec une dépendance locale, via des probabilités de transition

$$\mathbb{P}(X_i = 1 \mid X_{i-1}),$$

paramétrées (par exemple) par deux valeurs p_0 et p_1, correspondant typiquement à $\mathbb{P}(X_i = 1 \mid X_{i-1} = 0) = p_0$ et $\mathbb{P}(X_i = 1 \mid X_{i-1} = 1) = p_1$.

Notations de hachage et de charge

On note :
— n : le nombre d'éléments insérés ;
— ℓ : la dimension de sortie (en bits) ;
— $m = 2^\ell$: le nombre de seaux (*buckets*).
Pour chaque seau $j \in \{0, \dots, m-1\}$, on définit la charge

$$\mathrm{load}_j$$

comme le nombre d'éléments dont l'image par la fonction de hachage vaut j. La charge maximale est définie par

$$L_{\max} = \max_{0 \le j \le m-1} \mathrm{load}_j.$$

L'objectif expérimental du projet est d'étudier le comportement de $L_{\max}$ sous différentes distributions d'entrée.

3 Linear Hashing Model

3.1 Partie I

3.1.1 Définition de la fonction de hachage

Soit l'espace d'entrée

$$\mathbb{F}_2^u$$

et l'espace de sortie

$$\mathbb{F}_2^\ell.$$

On considère une matrice aléatoire

$$A \in \mathbb{F}_2^{\ell \times u}.$$

La fonction de hachage linéaire associée est définie par

$$h_A(x) = Ax \pmod 2,$$

où toutes les opérations sont effectuées modulo 2.

Le produit Ax correspond à un produit matrice–vecteur dans $\mathbb{F}_2$. La sortie $h_A(x)$ est donc un vecteur binaire de longueur ℓ.

3.1.2 Source d'aléa

Les coefficients de la matrice A sont générés indépendamment selon

$$A_{i,j} \sim \mathrm{Bernoulli}(1/2).$$

Ainsi, h_A est une application linéaire aléatoire. L'aléa du modèle provient :
— de la matrice de hachage A ;
— éventuellement des données d'entrée si celles-ci sont aléatoires.

3.1.3 Interprétation en seaux

La sortie

$$h_A(x) \in \mathbb{F}_2^\ell$$

peut être interprétée comme un entier dans

$$\{0, 1, \ldots, 2^\ell - 1\}.$$

On note

$$m = 2^\ell$$

le nombre total de seaux (buckets).

3.1.4 Définition de la charge

Soit n le nombre d'éléments insérés. Pour chaque seau j, on définit la charge

$$\text{load}_j = \left|\{i : h_A(x^{(i)}) = j\}\right|.$$

La charge maximale est alors définie par

$$L_{\max} = \max_{0 \le j < m} \text{load}_j.$$

3.1.5 Propriétés théoriques

La famille des fonctions de hachage linéaires sur $\mathbb{F}_2$ est connue pour être 2-universelle. Lorsque les entrées sont uniformes, les sorties sont proches d'une distribution uniforme sur les seaux.

Cependant, pour des entrées structurées (par exemple à poids fixé ou markoviennes), la distribution des images peut présenter des corrélations, ce qui peut influencer la charge maximale.

L'objectif de ce travail est d'étudier expérimentalement ce comportement sous différentes distributions d'entrée.

4 Experimental Setup

4.1 Partie I

4.1.1 Introduction

Pour évaluer les performances du hachage linéaire sous différentes structures d'entrée, nous générons d'abord une matrice de hachage aléatoire dans chaque essai

$$A \in \mathbb{F}_2^{\ell \times u}, \qquad A_{i,j} \sim \text{Bernoulli}(1/2),$$

et nous fixons la dimension de sortie à ℓ, ce qui correspond à

$$m = 2^\ell$$

seaux (*buckets*).

Ensuite, pour une distribution d'entrée $\mathcal{D}$ donnée, nous générons n vecteurs binaires

$$x^{(1)}, \ldots, x^{(n)} \sim \mathcal{D} \subseteq \mathbb{F}_2^u,$$

puis nous calculons pour chaque élément l'indice de seau

$$b_i = h_A(x^{(i)}) \in \{0, \ldots, m-1\}.$$

Nous comptons alors les charges load_j de chaque seau et nous enregistrons la charge maximale de l'essai :

$$L_{\max} = \max_{0 \le j < m} \text{load}_j.$$

Ce protocole est répété T fois (essais indépendants) afin d'obtenir une estimation empirique de la distribution de $L_{\max}$. Nous reportons typiquement la moyenne, ainsi que des quantiles (par ex. médiane et 90e percentile), et nous étudions l'évolution de ces statistiques en fonction des paramètres.

Pour quantifier l'influence de la structure des entrées, nous considérons deux familles de modèles :

— **Modèles à bits indépendants** : uniforme (Bernoulli(1/2)) et Bernoulli biaisée (Bernoulli(p)) ;

— **Modèles structurés / dépendants** : distribution à poids fixé et modèle markovien.

Sauf mention contraire, toutes les distributions sont testées avec les mêmes paramètres (u, ℓ, n, T), afin d'assurer une comparaison équitable. Lorsque cela est pertinent, une graine pseudo-aléatoire est fixée pour garantir la reproductibilité des résultats.

4.1.2 Procédure expérimentale

(a) Problème

Ky : il y a un section **(a)** avant **1**, peut-etre ne pas numeroter **(a)** ici, mettre que qques phrases introductives. Au cours des expériences, plusieurs difficultés sont apparues, liées à la fois aux hypothèses théoriques et aux contraintes de calcul.

1. Écart entre conditions théoriques et régime expérimental

Le résultat théorique étudié affirme que, pour $u \ge \ell \ge 1$, $m \le 2^u$ et $n = 2^\ell$, et pour tout $r \ge 6$:

$$\Pr_{h \sim \mathcal{H}} [M(S, h) \ge r \cdot \text{opt}(m, n)] \le \frac{49}{(r-2)^2}.$$

Observation : Lors de nos premières expériences, nous avons considéré des valeurs relativement petites de ℓ (typiquement $\ell \le 20$).

Empiriquement, nous avons observé que la probabilité

$$\Pr(M(S, h) \ge r \cdot \text{opt})$$

devient déjà négligeable pour des valeurs de r autour de 2.0, bien en dessous du seuil théorique $r \ge 6$.

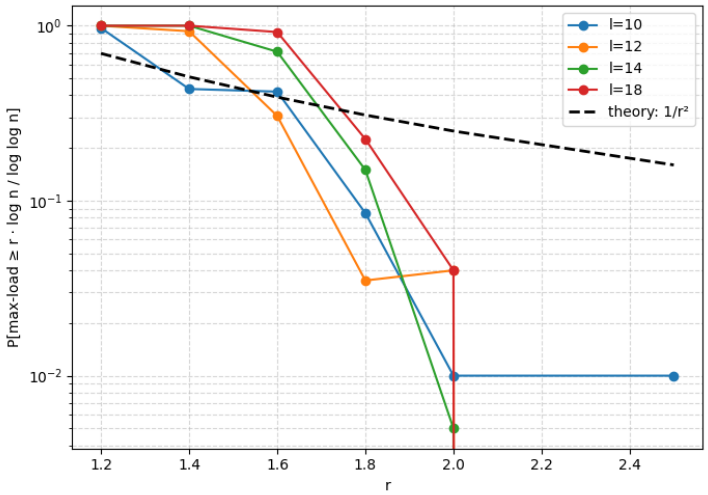

FIGURE 1 – Probabilité empirique en fonction de r pour $\ell \leq 20$.

Cela suggère que, dans ce régime, le comportement observé est plus favorable que la borne théorique.

Réflexion : Nous interprétons cette différence comme un effet de taille finie : les valeurs de ℓ utilisées ne sont pas suffisamment grandes pour atteindre le régime asymptotique sous-jacent à l'analyse théorique.

Pour se rapprocher du cadre du théorème, nous avons alors envisagé de fixer $\ell = 100$.

2. Difficultés computationnelles pour $\ell = 100$

Lorsque $\ell = 100$, le nombre de seaux devient

$$m = 2^{100},$$

ce qui pose deux problèmes majeurs.

(a) Complexité temporelle. Le calcul répété de Ax pour un grand nombre d'éléments et de trials devient extrêmement coûteux. Le temps d'exécution augmente significativement, rendant les simulations longues voire impraticables.

(b) Complexité du mémoire. Il est impossible de matérialiser explicitement 2^{100} seaux en mémoire. La mémoire requise dépasserait largement les capacités physiques disponibles.

Solution

(a) Complexité temporelle L'implémentation initiale reposait sur Python et NumPy pour effectuer le calcul du produit matrice–vecteur

$$h_A(x) = Ax \pmod 2.$$

Bien que NumPy soit efficace pour les calculs numériques, plusieurs facteurs limitaient les performances :

— un surcoût lié à l'interprétation Python, particulièrement lors de l'exécution répétée d'un grand nombre d'essais ;
— l'absence d'optimisation spécifique pour les opérations binaires ;
— une représentation mémoire utilisant 64 bits par coefficient au lieu d'un seul bit, augmentant la charge mémoire et les accès cache.

Du point de vue asymptotique, le protocole expérimental nécessite :

$$\mathcal{O}(Tnu\ell),$$

où T est le nombre d'essais, n le nombre d'éléments, u la dimension d'entrée et ℓ la dimension de sortie.

L'ordre de grandeur ne change pas avec le passage à C++. En revanche, la constante multiplicative est significativement réduite.

Optimisations :L'implémentation C++ repose sur :
— une représentation compacte des vecteurs binaires par blocs `uint64_t` (*bit-packing*) ;
— des opérations bit-à-bit (`AND`, `XOR`) permettant de traiter 64 bits simultanément ;
— la suppression du surcoût d'interprétation Python ;
— l'exécution parallèle des essais via le multi-threading.

En pratique, ces optimisations permettent une réduction substantielle du temps d'exécution, rendant possibles des expériences avec des valeurs plus grandes de ℓ et un nombre élevé de simulations.

(b) Complexité du mémoire Un second défi majeur concerne la complexité spatiale du protocole expérimental.

Dans le modèle théorique, le nombre de seaux est

$$m = 2^\ell.$$

Lorsque ℓ est modéré (par exemple $\ell \leq 20$), il est encore possible d'allouer un tableau de taille m pour stocker explicitement la charge de chaque seau.

Cependant, lorsque ℓ devient grand (par exemple $\ell = 100$), le nombre de seaux devient

$$m = 2^{100},$$

ce qui est totalement irréaliste en pratique. Même en supposant qu'un compteur occupe seulement 8 octets, la mémoire requise serait astronomique et largement au-delà des capacités physiques d'un système réel.

Lien vers le github :https ://github.com/C-killer/Observing-Linear-Hashing ?tab=readme-ov-file

Il est important de noter que, dans nos expériences, le nombre d'éléments insérés est $n \ll 2^\ell$. En conséquence, seuls un nombre limité de seaux sont effectivement occupés.

Optimisations : Plutôt que d'allouer un tableau complet de taille 2^ℓ, l'implémentation C++ adopte une stratégie dynamique :
— seuls les seaux effectivement observés sont stockés ;
— la structure de données maintient uniquement les charges pertinentes pour l'estimation de la charge maximale ;
— la mémoire utilisée dépend du nombre d'éléments insérés ou du nombre de compteurs maintenus, et non de 2^ℓ.

Ainsi, la complexité mémoire passe d'un ordre exponentiel théorique

$$\mathcal{O}(2^\ell)$$

à une complexité proportionnelle au nombre d'éléments traités, typiquement

$$\mathcal{O}(n)$$

ou même inférieure selon la structure employée.

Cette réduction est essentielle pour rendre possible l'étude expérimentale de valeurs élevées de ℓ, tout en respectant les contraintes matérielles.

5 Results and Profiling

Résultats pour le partie I

Après les optimisations décrites précédemment, nous avons obtenu un gain de performance allant d'un facteur $4\times$ à $70\times$ par rapport à l'implémentation initiale.

Ces améliorations ont permis d'explorer des valeurs plus élevées de ℓ et d'augmenter significativement le nombre d'essais réalisables.

Cependant, malgré ces optimisations, le régime $\ell = 100$ reste hors de portée en pratique.

Une estimation analytique du coût de calcul, basée sur la complexité

$$\mathcal{O}(Tnu\ell),$$

montre que le temps nécessaire pour exécuter un seul essai avec $\ell = 100$ serait astronomique (ordre de grandeur incompatible avec toute contrainte matérielle réaliste).

En comparaison, les expériences deviennent déjà très coûteuses pour $\ell \approx 20$, qui constitue la limite pratique atteignable dans notre cadre expérimental.

Ces observations suggèrent que la borne théorique sur la charge maximale, valable pour des paramètres asymptotiques et notamment pour $r \geq 6$, se situe dans un régime qui dépasse largement les capacités de simulation directe sur $\mathbb{F}_2$.

Autrement dit, bien que le résultat théorique soit valide, sa vérification expérimentale dans le régime ℓ très grand apparaît computationnellement irréalisable avec les ressources disponibles.

Cela met en évidence l'écart entre l'analyse asymptotique et les contraintes pratiques, et souligne la difficulté de valider empiriquement certaines bornes théoriques dans des dimensions élevées.

Ky : s'il n'y a pas encore des contenus pour les sections restantes (**Results and Profiling, Discussion, Conclusion**), mieux de les supprimer et les ajouter plus tard a la fin du projet.

6 Discussion

7 Conclusion